TABLE OF CONTENTS

1. Rationale 1.1 Problem Description & Background 1.2 Community Impact 1.3 Proposed Solution 1.4 Computational Thinking Process 1.5 SDG Alignment 1.6 Open Source Justification 1.7 Digital Public Goods Justification
2. Functional Modules
3. Expected Outputs
4. GitHub Evidence
5. References

1. RATIONALE

1.1 Problem Description & Background

Many local communities in Sierra Leone face significant challenges in education, particularly regarding access to public services and technology. Secondary schools in rural areas often lack the reliable internet infrastructure required to run modern, cloud-based educational management systems. Consequently, school administrators and teachers are forced to rely on fragile manual paper ledgers to record student scores and determine final term grades. This manual process is time-consuming, prone to human error during calculations, and leaves critical academic data highly vulnerable to physical damage or misplacement.

1.2 Community Impact

This problem directly impacts teachers, school administrators, and students in offline communities. By solving this issue, teachers will save hours previously spent manually calculating averages and determining pass/fail statuses. School administrators will benefit from a more organized and accurate record-keeping framework, which ultimately ensures that students receive fair and accurately calculated academic evaluations. The long-term sustainability of this project relies on its offline capabilities, ensuring it functions continuously regardless of power grid fluctuations or internet outages.

1.3 Proposed Solution

The proposed solution is an Offline Student Record System designed specifically for the Sierra Leonean secondary education framework. The system focuses on local data entry and processing without requiring web connectivity. It processes student names, collects scores across multiple subjects, calculates the total score and average, and automatically assigns a final term status (Pass or Fail) based on the calculated average.

1.4 Computational Thinking Process

Developing this logical design required applying computational thinking principles:

- **Decomposition:** The overall problem was broken down into smaller, manageable phases: collecting inputs, calculating totals and averages, determining the final status, and displaying the results.
- **Pattern Recognition:** Recognizing that score collection is a repetitive task, requiring the use of repetition structures (loops) to process an variable number of subjects.
- **Abstraction:** Focusing purely on the necessary logical steps (inputting scores and math calculations) while ignoring unnecessary complexities like graphic user interfaces.
- **Algorithm Design:** Creating a step-by-step sequence of instructions to navigate the data from input to the final output report.

1.5 SDG Alignment

This proposed system directly supports **Sustainable Development Goal 4 (Quality Education)**. By modernizing school administration tools and removing the barrier of internet dependency, the system ensures accurate, equitable tracking of student performance, which is a critical foundation for quality educational environments.

1.6 Open Source Justification

This system should be developed as Open Source Software so that local tech communities can modify, adapt, and distribute it freely. The primary benefit for Sierra Leone is that resource-constrained schools will not be burdened by prohibitive licensing fees. Furthermore, community developers can contribute by translating the software into local dialects or adding new modules, ensuring continuous, community-driven improvement.

1.7 Digital Public Goods (DPGs) Justification

This software qualifies as a Digital Public Good because it promotes accessibility, openness, and significant public benefit. By addressing a systemic barrier to education management, it serves as an open structural framework that the Ministry of Basic and Senior Secondary Education or local NGOs could adopt and scale nationwide.

2. FUNCTIONAL MODULES

The system logic is divided into distinct functional modules using modular design principles:

- **Input Module:** Prompts the user to enter the student's name and the total number of subjects.
- **Processing Module (Repetition):** Utilizes a loop to collect the individual score for each subject and calculate a running total.
- **Decision Module (Functions):** Utilizes a separate function containing an IF/ELSE decision structure to evaluate the average score. If the average is 50 or above, it returns "Pass"; otherwise, it returns "Fail".

- **Output Module:** Generates the final offline report displaying the student's name, total score, and final term status.

3. EXPECTED OUTPUTS

Below is a sample scenario demonstrating the expected system output:

Input Data:
Student Name: Mariatu Kamara
Number of Subjects: 4
Score 1: 55
Score 2: 60
Score 3: 45
Score 4: 50

System Processing: Total Score = 210
Average Score = 52.5
Status Decision = Pass

Expected Output Console:
--- Offline Term Report ---
Student: Mariatu Kamara
Total Score: 210
Term Final Grade: Pass

4. GITHUB EVIDENCE

- **GitHubRepositoryURL:**
https://github.com/osnatu2011/PROG101_Assignment1_AminataOKamara
- **Repository Screenshot:**

